Kubernetes RBAC Permission Denied Runbook

Document ID: RB-K8S-011 | Version: 1.0 | Last Updated: 2024-11-15

Category	kubernetes	**Severity**	P2 - High
Domain	Container Orchestration	**Est. Duration**	15-25 minutes
Tags	kubernetes, rbac, permission, serviceaccount, clusterrole, rolebinding, unauthorized		

Overview

RBAC Permission Denied errors (403 Forbidden) mean a ServiceAccount, user, or group is attempting an action not permitted by the current Role or ClusterRole bindings. This affects pod-to-API-server communication (operators, controllers, CI/CD pipelines) and kubectl access for human users.

Prerequisites

• kubectl with cluster-admin permissions for RBAC diagnosis

• Knowledge of which ServiceAccount the affected pod uses

• Access to application logs showing the 403 error

Response Steps

Step 1: Identify the Denied Action

Collect the exact RBAC error from pod logs or kubectl output, including the verb, resource, and namespace.

```
kubectl logs -n | grep -i 'forbidden\|403\|permission denied'
kubectl auth can-i -n --as=system:serviceaccount::
```

NOTE: The auth can-i command simulates what a ServiceAccount is allowed to do.

Step 2: Check ServiceAccount Used by Pod

Identify the ServiceAccount name bound to the affected pod.

```
kubectl get pod -n -o jsonpath='{.spec.serviceAccountName}'
kubectl get serviceaccount -n
```

Step 3: Check Existing RoleBindings

List all RoleBindings and ClusterRoleBindings for the ServiceAccount.

```
kubectl get rolebindings -n -o wide
kubectl get clusterrolebindings -o wide | grep
kubectl describe rolebinding -n
```

Step 4: Inspect the Role/ClusterRole

Examine the rules in the Role or ClusterRole to understand what is and is not permitted.

```
kubectl describe role -n
kubectl describe clusterrole
```

Step 5: Create or Update RoleBinding

Grant the missing permission by creating a new RoleBinding or patching the existing Role.

```
# Option A: bind to existing ClusterRole
kubectl create rolebinding --clusterrole= --serviceaccount=: -n
# Option B: create custom Role with specific verbs
kubectl create role --verb=get,list,watch --resource=pods -n
kubectl create rolebinding --role= --serviceaccount=: -n
```

NOTE: Follow least-privilege: grant only the exact verbs and resources needed, not wildcards.

Step 6: Verify Permission Granted

Re-run the auth can-i check and verify the pod now has the required access.

```
kubectl auth can-i -n --as=system:serviceaccount::
kubectl rollout restart deployment/ -n
kubectl logs -n | grep -i 'forbidden\|403' || echo 'No RBAC errors'
```

NOTE: SUCCESS: auth can-i returns 'yes' and pod logs show no more 403 errors.

Rollback Steps

Rollback Step 1: Remove Over-Permissive Binding

If a binding was granted too broadly, delete it and create a narrower one.

```
kubectl delete rolebinding -n
kubectl create role --verb=get,list --resource=pods -n
```

Rollback Step 2: Audit All RoleBindings

After any RBAC change, audit all bindings in the namespace.

```
kubectl get rolebindings,clusterrolebindings -A -o wide | grep
kubectl auth can-i --list --as=system:serviceaccount:: -n
```

Step Dependencies

Step	Depends On	Can Parallelize With
Step 1: Identify Denied Action	—	—
Step 2: Check ServiceAccount	Step 1	—
Step 3: Check RoleBindings	Step 2	—
Step 4: Inspect Role/ClusterRole	Step 3	—
Step 5: Create/Update RoleBinding	Step 3, Step 4	—
Step 6: Verify Permission	Step 5	—
